serviceOrchestrationPushManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationPushManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationPushManagement` service of the DynamicServiceOrchestration system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation subscribe . . . 4
3.2 operation unsubscribe . . . 6
3.3 operation trigger . . . 6
3.4 operation query . . . 7

4 Data Models **9**
4.1 struct ErrorMessage . . . 9
4.2 struct MetadataRequirement . . . 9
4.3 struct OrchestrationJob . . . 9
4.4 struct OrchestrationNotifyInterface . . . 9
4.5 struct OrchestrationPushJobListResponse . . . 10
4.6 struct OrchestrationPushTrigger . . . 10
4.7 struct OrchestrationRequest . . . 10
4.8 struct OrchestrationServiceRequirement . . . 10
4.9 struct OrchestrationSubscriptionListRequest . . . 11
4.10 struct OrchestrationSubscriptionListResponse . . . 11
4.11 struct OrchestrationSubscriptionQueryRequest . . . 11
4.12 struct OrchestrationSubscriptionRequest . . . 11
4.13 struct OrchestrationSubscriptionResponse . . . 12
4.14 struct Page . . . 12
4.15 struct QoSRequirement . . . 12
4.16 Primitives . . . 13

5 Go and Rust implementation variants **14**
5.1 Go implementation . . . 14
5.2 Rust implementation . . . 14

6 References **14**

7 Revision History **15**
7.1 Amendments . . . 15
7.2 Quality Assurance . . . 15

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `serviceOrchestrationPushManagement` service interface of the Dynamic-ServiceOrchestration system, which provides operator management of push orchestration: subscriptions made on behalf of consumers, and triggering of push orchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationPushManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationPushManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationPushManagement` service interface in detail. Both in terms of its operations (Section 3) and its information model (Section 4).

2.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/orchestration/mgmt/push`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`subscribe`	POST	`/subscribe`
`unsubscribe`	DELETE	`/unsubscribe`
`trigger`	POST	`/trigger`
`query`	POST	`/query`

3 Service Operations

The interfaces of the `serviceOrchestrationPushManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationPushManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

3.1 POST /serviceorchestration/orchestration/mgmt/push/subscribe

Operation: subscribe
Input: OrchestrationSubscriptionListRequest
Output: OrchestrationSubscriptionListResponse

Creates subscriptions on behalf of target systems.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/subscribe HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "subscriptions": [
    {
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
```

```
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "duration": 3600
    }
  ]
}
```

Listing 1: An subscribe invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ownerSystemName": "TemperatureDisplay",
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
```

```
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "expiredAt": "2026-10-22T12:00:00Z",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An subscribe response.

3.2 DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe?ids={ids}

Operation: unsubscribe
Input: Array<String>

Removes subscriptions by identifier.
Query parameters: *ids* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe?ids=<ids> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An unsubscribe invocation.

3.3 POST /serviceorchestration/orchestration/mgmt/push/trigger

Operation: trigger
Input: OrchestrationPushTrigger
Output: OrchestrationPushJobListResponse

Starts push orchestration for the selected subscriptions (by target system or identifier); returns the started jobs.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/trigger HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 4: An trigger invocation.

```
{
  "jobs": [
    {
```

```
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "status": "DONE",
      "type": "IPV4",
      "requesterSystem": "TemperatureDisplay",
      "targetSystem": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "subscriptionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "message": "1 local result: TemperatureSensor",
      "createdAt": "2026-10-22T12:00:00Z",
      "startedAt": "2026-10-22T12:00:00Z",
      "finishedAt": "2026-10-22T12:00:00Z"
    }
  ]
}
```

Listing 5: An trigger response.

3.4 POST /serviceorchestration/orchestration/mgmt/push/query

Operation: query
Input: OrchestrationSubscriptionQueryRequest
Output: OrchestrationSubscriptionListResponse

Returns a page of subscriptions matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/query HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ownerSystems": [
    "string"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ]
}
```

Listing 6: An query invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ownerSystemName": "TemperatureDisplay",
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
```

```
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "expiredAt": "2026-10-22T12:00:00Z",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 7: An query response.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 4.16, which are used to represent things like hashes, identifiers and texts.

4.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.2 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

4.3 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

4.4 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

4.5 struct OrchestrationPushJobListResponse

Push jobs started by a trigger.

Object Field	Value Type	Description
"jobs"	Array<OrchestrationJob>	Orchestration jobs started by the trigger.

4.6 struct OrchestrationPushTrigger

Selects the subscriptions to execute now.

Object Field	Value Type	Description
"targetSystems"	Array<String>	Consumers whose subscriptions are executed.
"subscriptionIds"	Array<String>	Subscriptions to execute.

4.7 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

4.8 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

4.9 struct OrchestrationSubscriptionListRequest

Subscriptions to create on behalf of consumers.

Object Field	Value Type	Description
"subscriptions"	Array<OrchestrationSubscription Request>	Subscriptions to create.

4.10 struct OrchestrationSubscriptionListResponse

List of subscriptions.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSubscription Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

4.11 struct OrchestrationSubscriptionQueryRequest

Paged filter for subscriptions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ownerSystems"	Array<String>	Subscription owner names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.

4.12 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

4.13 struct OrchestrationSubscriptionResponse

A stored subscription.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the subscription.
"ownerSystemName"	String	Name of the system that created the subscription.
"targetSystemName"	String	Name of the consumer system the push orchestration is made for.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"expiredAt"	DateTime	UTC time when the subscription expires.
"createdAt"	DateTime	UTC time of creation.

4.14 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

4.15 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

4.16 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8441 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8441". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-dynamic`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/DynamicServiceOrchestration`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	